

CHAPITRE III

MÉTHODOLOGIE ET CONCEPTION DU SYSTÈME

Ce chapitre expose avec précision les choix méthodologiques, les outils technologiques retenus et l'architecture de conception du système de recommandation hybride emploi-compétences pour le marché camerounais. Chaque décision est rigoureusement justifiée du point de vue théorique et pratique.

3.1 POSITIONNEMENT METHODOLOGIQUE GENERAL

Ce mémoire s'inscrit dans le domaine de la **science des données appliquée** et du traitement automatique du langage naturel (TALN). Il adopte une démarche de recherche-développement (R&D) de type constructiviste : nous ne nous contentons pas d'observer un phénomène existant, mais nous concevons, implémentons et évaluons un artefact numérique — le système de recommandation — dont l'utilité répond à un problème réel et documenté du marché du travail camerounais.

Le problème central est une inadéquation structurelle entre l'offre et la demande de compétences, que l'on qualifie de *skill mismatch*. Ce phénomène se manifeste de deux façons : d'une part, des candidats possèdent des compétences qui ne correspondent pas aux besoins exprimés par les employeurs (inadéquation qualitative) ; d'autre part, des offres publiées restent non pourvues malgré un vivier de candidats théoriquement qualifiés (inadéquation informationnelle). Les systèmes de recherche d'emploi existants, fondés sur la correspondance de mots-clés, sont structurellement incapables de traiter cette complexité sémantique.

Hypothèse centrale de la recherche

Un système hybride combinant (1) un graphe de connaissances structurant les entités et leurs relations sémantiques, (2) un module de similarité vectorielle dense fondé sur les embeddings, et (3) un LLM capable d'extraire, d'enrichir et de justifier les correspondances, produit des recommandations significativement plus pertinentes qu'une approche mono-source, aussi bien du point de vue de la précision (precision@k) que de la couverture (recall) et de l'explicabilité.

3.1.1 Paradigme méthodologique : la triangulation des sources de connaissance

La spécificité du contexte camerounais impose une stratégie de triangulation des sources de données. Contrairement aux marchés du travail européens ou nord-américains, il n'existe pas de référentiel national de compétences unifié, exhaustif et nativement numérique. Notre démarche consiste donc à croiser trois couches de connaissance complémentaires :

1. La couche **européenne universelle** : le référentiel ESCO (European Skills, Competences, Qualifications and Occupations) de la Commission Européenne, disponible en français, fournit une taxonomie de 13 890 compétences et 3 008 métiers avec leurs relations. Il constitue notre ossature sémantique principale.
2. La couche **camerounaise des métiers** : la Nomenclature Camerounaise des Métiers, Emplois et Professions (MEPC, 2013) de l'INS fournit 8 grands groupes, 44 sous-groupes et 209 groupes de base alignés sur la Classification Internationale Type des Professions (CITP-08). Elle ancre le système dans la réalité institutionnelle locale.
3. La couche **camerounaise des formations** : la Nomenclature Camerounaise des Formations (NCF, 2017), également produite par l'INS, codifie 9 niveaux et 201 domaines de formation détaillés. Elle permet d'associer chaque profil de candidat à un niveau formel et un domaine disciplinaire précis.

Cette triangulation est un apport scientifique original : à notre connaissance, aucun système de matching emploi-compétences existant ne mobilise simultanément ESCO, MEPC et NCF dans un graphe de connaissances intégré pour le contexte camerounais.

3.2 SOURCES DE DONNEES ET CONSTITUTION DU CORPUS

3.2.1 Description des jeux de données

Le système repose sur deux types de données empiriques complémentaires : des données d'offres d'emploi publiées sur des plateformes numériques, et des données de demandeurs d'emploi issues d'opérations de collecte terrain. Ces deux sources sont enrichies par les référentiels terminologiques décrits ci-dessus.

3.2.1.1 Données d'offres d'emploi

Le premier jeu de données compile des offres d'emploi publiées sur des portails actifs au Cameroun (Emploi.cm, Rekrute Afrique, LinkedIn, etc.). Chaque offre est structurée autour de dix-sept attributs : Source, Lien/Référence, Titre du Poste, Employeur, Type d'Entreprise, Pays, Ville/Région, Secteur d'Activité, Groupe de Contrat, Type de Contrat, Niveau d'Expérience, Niveau d'Études, Compétences/Skills (champ textuel libre), Date de Publication, Mois, Trimestre et Détails de l'Annonce.

L'attribut « Compétences/Skills » est particulièrement hétérogène : les employeurs y inscrivent des termes libres (« maîtrise d'Excel », « esprit d'analyse », « Python ») qui doivent être normalisés vers les URIs ESCO lors du pré-traitement. C'est précisément l'une des fonctions du module LLM décrit en section 3.5. L'attribut « Détails de l'Annonce » est un champ textuel long (de quelques lignes à plusieurs paragraphes) exploité comme source primaire pour l'extraction de compétences implicites.

3.2.1.2 Données de demandeurs d'emploi

Le second jeu de données porte sur des profils de demandeurs d'emploi et couvre quatorze attributs : Secteur d'activité, Objectif, Diplôme, Genre, Département, Mois, Trimestre, Niveau d'Étude (codifiable selon la NCF), Qualification Métier (codifiable selon la MEPC), Secteur Métier, Filière/Spécialité, Secteur Demandé, Métier Visé/Qualification Visée et Mobilité Géographique.

La richesse de ce jeu de données réside dans la co-présence d'attributs structurés (niveau d'étude codifiable en NCF, qualification métier codifiable en MEPC) et d'attributs sémantiques libres (objectif, filière). Cette dualité structure/texte justifie l'architecture hybride : les attributs structurés alimentent le graphe Neo4j, les textes libres alimentent le module d'embedding pgvector.

3.2.2 Référentiels terminologiques : ESCO, MEPC, NCF

3.2.2.1 ESCO — base sémantique internationale

ESCO (European Skills, Competences, Qualifications and Occupations) est le multilingual classification system officiel de la Commission Européenne, publié sous licence CC-BY 4.0. Sa version 1.2.0 (2023), exploitée dans sa déclinaison française, couvre :

-
-
-
-

Les 18 fichiers CSV fournis ont été analysés dans leur intégralité. Les fichiers structurellement centraux pour notre système sont : `skills_fr.csv` (entités compétences), `occupations_fr.csv` (entités métiers), `occupationSkillRelations_fr.csv` (27 Mo — la plus grande table, 74 957 relations), `skillsHierarchy_fr.csv` (hiérarchie Bigger/Narrower entre compétences), `ISCOGroups_fr.csv` (groupes ISCO de 1 à 4 chiffres) et les six collections thématiques (digital, green, language, research, transversal, digComp).

3.2.2.2 Justification du choix d'ESCO comme ossature sémantique

ESCO est retenu comme référentiel principal — et non O*NET (américain) ni le ROME (français) — pour trois raisons convergentes :

- **Couverture linguistique** : ESCO est natif multilingue (27 langues dont le français), ce qui élimine les erreurs de traduction dans un corpus camerounais francophone.
- **Alignement institutionnel** : ESCO est aligné sur ISCO-08, qui constitue lui-même le référentiel des codes CITE-08 utilisés dans la MEPC camerounaise. Cette compatibilité structurelle permet le pontage direct entre ESCO et MEPC via les codes ISCO.
- **Accès programmatique** : ESCO fournit une API REST et des dumps CSV complets sous licence libre, compatibles avec un pipeline Python/Neo4j.

3.3 ARCHITECTURE GENERALE DU SYSTEME

L'architecture proposée est modulaire et stratifiée en cinq couches fonctionnelles distinctes, chacune remplissant un rôle précis dans le pipeline de recommandation. Cette stratification répond à un principe fondamental d'ingénierie logicielle : la séparation des préoccupations (separation of concerns), qui garantit la maintenabilité, la testabilité et l'évolutivité du système.

Couche	Module	Rôle fonctionnel	Technologies
C1	Ingestion et normalisation	Nettoyage, standardisation, mapping vers les référentiels (ESCO/MEPC/NCF)	<i>Python · Pandas · Spacy · LLM (GPT/Mistral)</i>
C2	Graphe de connaissances	Stockage des entités et relations sémantiques ; traversées multi-hop	<i>Neo4j 5.x · Cypher</i>
C3	Représentation vectorielle	Embedding des textes ; recherche ANN ; stockage dense	<i>pgvector · Sentence-Transformers · PostgreSQL</i>
C4	Enrichissement LLM	Extraction de compétences implicites ; génération de roadmap ; explication	<i>LLM via API (GPT-4o / Mistral Large)</i>
C5	Moteur de recommandation	Calcul du score hybride ; filtrage collaboratif ; classement final	<i>Python · FastAPI · Neo4j + pgvector</i>

Tableau 3.1 — Architecture en cinq couches du système hybride de recommandation

3.4 CONCEPTION DU GRAPHE DE CONNAISSANCES (NEO4J)

3.4.1 Pourquoi un graphe de connaissances ?

Un graphe de connaissances (Knowledge Graph) est une représentation formelle d'entités du monde réel et de leurs interrelations sous la forme d'un triplet (sujet, prédicat, objet). Ce paradigme est intrinsèquement adapté au problème du matching emploi-compétences pour plusieurs raisons fondamentales :

- **Hétérogénéité des entités** : notre domaine implique des entités de natures radicalement différentes — candidats, offres d'emploi, compétences techniques, compétences transversales, métiers, niveaux de formation, secteurs économiques, localités — dont les interactions ne peuvent être représentées efficacement dans un schéma relationnel tabulaire sans une multiplication de jointures coûteuses.
- **Richesse des relations** : le matching emploi-compétences est fondamentalement un problème de traversée de graphe. Un candidat qui possède la compétence « analyse statistique » peut correspondre à une offre requérant « fouille de données » si ces deux compétences sont liées par la relation PLUS_LARGE_QUE dans la hiérarchie ESCO. Cette inférence transitive est triviale dans un graphe (chemin de longueur 2) et très coûteuse en SQL (jointure récursive).
- **Raisonnement multi-hop** : la question « Quels candidats ayant une formation en NCF 05503 (statistiques) sont pertinents pour une offre de data analyst ? » implique de traverser NCF → Compétences → Métier → Offre en un seul parcours Cypher, ce qui serait impossible ou extrêmement complexe en SQL.
- **Explicabilité naturelle** : chaque chemin dans le graphe est une explication en soi : le jury (et l'utilisateur) peut littéralement « lire » la chaîne de raisons qui conduit à une recommandation, ce qui répond à l'exigence d'explicabilité (explainability) des systèmes d'IA.

3.4.2 Justification du choix de Neo4j

Parmi les systèmes de gestion de bases de données orientés graphe disponibles, Neo4j (v5.x Community/Enterprise) a été retenu face à ses concurrents directs (Amazon Neptune, ArangoDB, JanusGraph, TigerGraph). Le tableau comparatif ci-dessous détaille les critères de décision :

Critère	Neo4j	Amazon Neptune	ArangoDB	JanusGraph
Langage de requête	Cypher (déclaratif, intuitif)	SPARQL + Gremlin (complexe)	AQL (hybride)	Gremlin (verbeux)
Performance traversée	Très haute (index natifs)	Haute (cloud-dépendant)	Haute	Moyenne

Critère	Neo4j	Amazon Neptune	ArangoDB	JanusGraph
Déploiement local	Oui — libre sans cloud	Non — AWS uniquement	Oui	Oui (complex)
Intégration Python	py2neo, neo4j-driver	Boto3 uniquement	arango-driver	Limitée
Maturité / Communauté	Très élevée (2007, leader)	Élevée	Élevée	Moyenne
APOC + GDS plugins	Oui — natif	Non	Partiel	Non
Coût (contexte mémoire)	Gratuit (Community Ed.)	Payant (AWS)	Gratuit	Gratuit mais complexe
Outil de visualisation	Neo4j Browser + Bloom	Neptune Workbench	ArangoDB UI	Gephi (externe)

Tableau 3.2 — Comparaison des SGBD orientés graphe pour le système de recommandation

Neo4j s'impose par trois avantages décisifs dans notre contexte : premièrement, le langage Cypher est déclaratif et lisible — un chercheur peut lire une requête de traversée comme une phrase en langage naturel, ce qui est fondamental pour la documentation et l'audit scientifique du système ; deuxièmement, la disponibilité gratuite de l'édition Community et du plugin APOC (Awesome Procedures on Cypher), qui fournit des procédures d'import CSV massif en batch, de calcul de similarité et de manipulation de collections ; troisièmement, l'écosystème Python mature (neo4j-driver officiel, py2neo) qui s'intègre naturellement dans le pipeline de traitement des données.

3.4.3 Schéma du graphe — nœuds et propriétés

3.4.3.1 Nœuds principaux

Le graphe comporte 16 types de nœuds répartis en quatre familles fonctionnelles. Chaque type de nœud est associé à une contrainte d'unicité (UNIQUE constraint) sur sa clé primaire, ce qui garantit l'idempotence des opérations d'ingestion (propriété essentielle lors des rechargements incrémentaux du graphe).

Famille	Type de nœud	Clé primaire	Propriétés notables
Acteurs	Candidat	id (UUID)	objectif, diplome, mobilite_geo, ncf_niveau, mepc_base
Acteurs	OffreEmploi	id (UUID)	titre_poste, secteur_activite, ncf_niveau, date_publication
Compétences	Compétence	conceptUri (ESCO)	preferredLabel, skillType, pillar, isDigital, isGreen
Compétences	GroupeCompétences	conceptUri (ESCO)	preferredLabel, niveau

Famille	Type de nœud	Clé primaire	Propriétés notables
Métiers	Métier	conceptUri (ESCO)	preferredLabel, iscoCode, mepc_base, ncf_domdet, naceCode
Métiers	GroupeISCO	code (1-4 chiffres)	preferredLabel, niveau
MEPC	GrandGroupeMEPC	code (1 chiffre)	intitule, codes_citp
MEPC	SousGroupeMEPC	code (2 chiffres)	intitule, codes_citp
MEPC	GroupeBaseMEPC	code (4 chiffres)	intitule, codes_citp
NCF	NiveauFormationNCF	code (1-9)	intitule, notes_explicatives
NCF	GrandDomaineNCF	code (2 chiffres)	intitule, explication
NCF	DomaineSpécialiséNCF	code (3 chiffres)	intitule, explication
NCF	DomaineDétailléNCF	code (5 chiffres)	intitule, explication
Contexte	Secteur	label	—
Contexte	Employeur	nom	type_entreprise
Contexte	Localisation	ville + pays	—

Tableau 3.3 — Types de nœuds du graphe de connaissances et leurs propriétés clés

3.4.3.2 Relations structurantes

Le graphe modélise 22 types de relations, catégorisées en quatre classes : les relations cœur (core), les relations hiérarchiques (hiérarchie), les relations de contexte (contexte) et les relations analytiques calculées (analytique). Cette classification permet de distinguer les relations statiques (issues des référentiels et figées après l'ingestion initiale) des relations dynamiques (calculées et mises à jour lors de chaque cycle de recommandation).

La relation la plus critique du graphe est **NECESSITE** (Métier → Compétence), dérivée du fichier `occupationSkillRelations_fr.csv` d'ESCO. Elle porte deux propriétés déterminantes pour le calcul du skill gap : `relationType` (essentielle ou optionnelle) et `skillType` (knowledge ou skill/compétence). Une compétence essentielle manquante pénalise davantage le score hybride qu'une compétence optionnelle manquante, ce qui reproduit le raisonnement d'un recruteur réel.

La relation **SIMILAIRE_A** (Compétence → Compétence) est le pont entre Neo4j et pgvector : elle est générée programmatiquement en exécutant des requêtes ANN (Approximate Nearest Neighbor) dans pgvector sur les embeddings de compétences, puis en persistant les paires dont le score cosinus dépasse un seuil $\theta = 0.82$. Cette relation matérialisée permet ensuite des traversées Cypher ultra-rapides pour trouver des compétences sémantiquement proches sans réinterroger pgvector à chaque requête.

3.5 MODULE DE SIMILARITE SEMANTIQUE (PGVECTOR)

3.5.1 Fondements théoriques : embeddings et similarité cosinus

Un embedding est une représentation d'un objet textuel (mot, phrase, document) sous forme d'un vecteur dense dans un espace de haute dimension $\mathbb{R}^n$, où les propriétés sémantiques sont encodées dans la géométrie de cet espace : deux textes sémantiquement proches ont des vecteurs qui pointent dans des directions voisines. Ce paradigme, fondé sur l'hypothèse distributionnelle de Harris (« a word is characterized by the company it keeps »), est à la base des modèles de langage modernes depuis Word2Vec (2013) jusqu'aux Transformer-based encoders.

La similarité entre deux vecteurs u et v est mesurée par le cosinus de l'angle qu'ils forment : $\cos(u,v) = (u \cdot v) / (\|u\| \cdot \|v\|)$. Cette métrique est invariante à la norme des vecteurs, ce qui la rend robuste face aux variations de longueur des textes — une description d'offre de trois paragraphes et un intitulé de poste de trois mots peuvent être comparés sans biais de longueur.

3.5.2 Choix du modèle d'embedding

Le choix du modèle d'embedding est la décision technique la plus impactante sur la qualité des recommandations. Plusieurs familles de modèles ont été évaluées :

Modèle	Dimension	Langue	Qualité FR	Coût	Justification
paraphrase-multilingual-mpnet-base-v2	768	50 langues	Excellente	Gratuit (local)	RETENU : meilleur équilibre qualité/coût pour le français
text-embedding-3-small (OpenAI)	1536	Multilingue	Excellente	Payant (\$)	Alternative cloud si ressources disponibles
CamemBERT-large	1024	Français uniquement	Très bonne	Gratuit (local)	Monolingue — limite l'intégration ESCO en anglais
sentence-transformers/LaBSE	768	109 langues	Bonne	Gratuit (local)	Moins performant sur le français que mpnet
fr-sentence-bert (DSF)	768	Français uniquement	Bonne	Gratuit (local)	Corpus d'entraînement limité, peu de recul

Tableau 3.4 — Comparaison des modèles d'embedding pour le corpus francophone

Le modèle **paraphrase-multilingual-mpnet-base-v2** (Sentence-Transformers / UKP Lab) est retenu pour quatre raisons convergentes : (1) il est entraîné spécifiquement pour la similarité

sémantique de phrases et non pour la classification, ce qui est exactement notre usage ; (2) sa dimension de 768 correspond à un bon compromis entre expressivité sémantique et coût de stockage/calcul (une table de 100 000 vecteurs $\times$ 768 floats = 300 Mo, gérable localement) ; (3) son caractère multilingue permet de traiter simultanément les textes en français (offres, candidats, ESCO français) et en anglais (labels ESCO originaux) sans avoir à maintenir deux modèles distincts ; (4) il est entièrement local — il tourne sur CPU ou GPU — ce qui garantit la confidentialité des données et élimine les coûts variables d'API.

3.5.3 Architecture pgvector — justification du choix PostgreSQL/pgvector

pgvector est une extension open-source pour PostgreSQL qui ajoute un type de données VECTOR(n) et deux algorithmes de recherche de voisins approximatifs : IVFFlat (Inverted File with Flat quantization) et HNSW (Hierarchical Navigable Small World). Ce choix s'oppose à des alternatives comme Pinecone, Weaviate, Milvus ou Chroma.

Décision d'architecture : pourquoi pgvector et non une base vectorielle native ?

Les bases vectorielles natives (Pinecone, Weaviate, Milvus) sont optimisées pour un seul usage — la recherche ANN — mais créent une fragmentation de l'écosystème : les métadonnées des offres et des candidats devraient être stockées ailleurs, introduisant des jointures inter-systèmes coûteuses. pgvector offre l'ANN dans PostgreSQL, qui héberge déjà nos tables de métadonnées (skill_gap, recommandations). Une seule requête SQL peut donc filtrer par métadonnées ET classer par similarité vectorielle en un seul passage. Ce paradigme de co-localisation réduit drastiquement la latence des requêtes hybrides.

3.5.3.1 Index HNSW vs IVFFlat

Deux algorithmes d'indexation sont disponibles dans pgvector. L'index HNSW (Hierarchical Navigable Small World) est retenu pour notre système :

- **HNSW** : recall élevé (>95% pour les paramètres retenus $m=16$, $ef_construction=64$), temps de construction plus lent mais requêtes quasi-instantanées (<10ms pour $k=20$). Adapté à notre cas où le corpus est relativement stable (rechargements peu fréquents) et où la latence des requêtes doit être minimale.
- **IVFFlat** : construction rapide, mais recall inférieur à HNSW pour un même budget de listes (nlist). Adapté aux corpus qui évoluent très fréquemment. Non retenu.

Les paramètres HNSW retenus ($m=16$, $ef_construction=64$) résultent d'un compromis : m contrôle le nombre de liens par nœud dans le graphe navigable (plus élevé = meilleur recall, plus d'espace) ; $ef_construction$ contrôle l'horizon de recherche lors de la construction (plus élevé = meilleure qualité d'index, construction plus lente). Ces valeurs correspondent aux recommandations pgvector pour les cas d'usage avec recall prioritaire sur la vitesse de build.

3.5.4 Construction du texte à embedder (text_to_embed)

Le champ text_to_embed est la décision la plus fine et la plus impactante sur la qualité sémantique du système. Il détermine ce que le modèle « comprend » de chaque entité. La stratégie retenue est la concaténation structurée des attributs les plus sémantiquement riches, différente selon le type d'entité :

- **Compétence ESCO** : preferredLabel + altLabels (synonymes) + description. Les synonymes (altLabels) sont capitaux : ils capturent les variations terminologiques entre employeurs (« analyse de données » ≈ « data analysis » ≈ « analytics »).
- **Métier ESCO** : même structure que la compétence, avec en plus la description longue qui mentionne les tâches typiques du poste.
- **Offre d'emploi** : titre_poste + secteur_activite + compétences brutes + détails de l'annonce. L'inclusion des détails longs est délibérée : ils contiennent les compétences implicites que le LLM n'a pas encore extraites.
- **Candidat** : objectif + qualification_metier + filiere_specialite + metier_vise. La concaténation de l'objectif (texte libre) et des attributs structurés produit un vecteur qui capture à la fois l'intention et la formation formelle.

3.6 MODULE D'ENRICHISSEMENT ET D'EXTRACTION PAR LLM

3.6.1 Rôle fonctionnel du LLM dans le pipeline

Le grand modèle de langage (Large Language Model, LLM) joue trois rôles distincts et non interchangeables dans notre système. Il est important de préciser que le LLM n'est pas le composant central du système de recommandation — ce rôle appartient à l'architecture hybride graphe/vecteur — mais qu'il est un enrichisseur indispensable qui opère en amont (extraction) et en aval (génération) du pipeline :

Rôle 1 — Extraction de compétences (NER spécialisé)

Les annonces d'emploi contiennent des compétences exprimées en langage naturel non normalisé (« bonne maîtrise des outils bureautiques », « capacité à travailler en équipe multiculturelle », « 3 ans d'expérience en gestion de projet »). Le LLM est utilisé comme extracteur de compétences nommées (Named Entity Recognition) puis comme mappeur vers les URIs ESCO. Le prompt est conçu pour produire un JSON structuré :

INSTRUCTION : Extrais les compétences de l'annonce suivante. Pour chaque compétence, retourne son label normalisé et l'URI ESCO la plus proche (format : <http://data.europa.eu/esco/skill/...>).
Retourne uniquement du JSON valide.

Ce prompt en deux étapes (extraction puis normalisation ESCO) est délibéré : demander directement une URI ESCO hallucinatoire est un risque connu des LLM. L'approche retenue consiste à extraire le label libre, puis à calculer la similarité cosinus entre l'embedding du label extrait et les embeddings de tous les preferredLabel ESCO pour identifier l'URI correcte — le LLM propose, pgvector valide.

Rôle 2 — Mapping NCF/MEPC (normalisation des profils candidats)

Les attributs `qualification_metier` et `filiere_specialite` des candidats sont des textes libres (« Technicien en informatique de gestion », « Master en statistiques appliquées »). Le LLM est utilisé pour les mapper vers les codes MEPC et NCF appropriés, en exploitant les descriptions textuelles chargées en contexte (few-shot prompting avec exemples tirés des nomenclatures).

Rôle 3 — Génération de roadmap explicative

C'est le rôle le plus visible pour l'utilisateur final. Après le calcul du skill gap (liste des compétences manquantes, partielles et acquises), le LLM génère une roadmap personnalisée structurée : pour chaque compétence manquante critique, il suggère un parcours de formation ancré dans les domaines NCF identifiés dans le graphe. Le prompt reçoit en contexte : le profil du candidat, les détails de l'offre, la liste annotée des écarts, et les chemins Neo4j (DomaineNCF → Compétence → Métier) retrouvés par traversée du graphe.

3.6.2 Choix du LLM et stratégie de prompting

Deux LLMs sont ciblés selon le contexte de déploiement : GPT-4o (OpenAI) pour les environnements cloud avec budget, et Mistral Large (ou Mixtral 8x7B en local via Ollama) pour les déploiements souverains. Les prompts sont conçus selon les principes du RAG (Retrieval-Augmented Generation) : le LLM ne génère pas de connaissances de mémoire mais raisonne exclusivement sur les données fournies en contexte, ce qui élimine le risque d'hallucination sur les référentiels ESCO/MEPC/NCF.

3.7 MOTEUR DE RECOMMANDATION HYBRIDE

3.7.1 Définition formelle du score hybride

Le score hybride $S(c, o)$ associé à une paire (candidat c , offre d'emploi o) est défini comme une combinaison linéaire convexe de trois composantes :

$$S(c, o) = \alpha \cdot S_{\text{sém}}(c, o) + \beta \cdot S_{\text{graph}}(c, o) + \gamma \cdot S_{\text{collab}}(c, o)$$

avec $\alpha + \beta + \gamma = 1$ (contrainte de convexité garantissant que $S \in [0, 1]$). Les pondérations initiales retenues sont $\alpha = 0.40$, $\beta = 0.35$, $\gamma = 0.25$, calibrables empiriquement lors de la phase d'évaluation.

3.7.2 Composante sémantique $S_{\text{sém}}(c, o)$

La composante sémantique mesure la proximité vectorielle entre le profil du candidat et l'offre d'emploi dans l'espace des embeddings. Elle est calculée via pgvector :

$$S_{\text{sém}}(c, o) = 1 - (e_c \oslash e_o) = \cos(e_c, e_o)$$

où e_c et e_o sont respectivement les vecteurs d'embedding du candidat et de l'offre, et $\oslash$ désigne l'opérateur de distance cosinus implémenté dans pgvector. Cette composante capture la similarité globale entre les intentions du candidat (objectif, filière, métier visé) et le contexte de l'offre (titre, secteur, description).

3.7.3 Composante graphe $S_{\text{graph}}(c, o)$

La composante graphe est calculée par traversée multi-hop du graphe Neo4j. Elle quantifie la correspondance structurelle entre le profil du candidat et les exigences de l'offre, en tenant compte de la hiérarchie des compétences ESCO. Elle repose sur deux sous-scores :

- **Score de skill gap direct** : ratio des compétences ESCO directement acquises par le candidat sur les compétences requises par l'offre. $S_{\text{gap_direct}} = |\text{skill}(c) \cap \text{skill}(o)| / |\text{skill}(o)|$.
- **Score de skill gap indirect** : pénalisé par la distance de traversée dans la hiérarchie ESCO (PLUS_LARGE_QUE). Une compétence parente (niveau 1 de distance) est créditée à 0.7, une compétence grand-parent (niveau 2) à 0.4, au-delà de 2 niveaux elle n'est pas créditée.

$$S_{\text{graph}}(c, o) = w_1 \cdot S_{\text{gap_direct}} + w_2 \cdot S_{\text{gap_indirect}} + w_3 \cdot S_{\text{niveau_ncf}}$$

où S_niveau_ncf pénalise les offres dont le niveau de formation requis dépasse significativement le niveau NCF certifié du candidat (un candidat de niveau 7 postulant à une offre niveau 9 reçoit un bonus moindre qu'une offre niveau 7-8).

3.7.4 Composante collaborative $S_collab(c, o)$

La composante collaborative exploite les patterns de postulation observés dans l'historique des interactions (candidats ayant des profils similaires et les offres qu'ils ont consultées/postulées). Elle est implémentée selon une approche de filtrage collaboratif basée sur les items (item-based CF) : deux offres sont considérées similaires si elles ont été postulées par des ensembles de candidats similaires. Cette composante complète les deux précédentes en capturant des correspondances non modélisables a priori par le graphe ou les embeddings — par exemple, des offres dans un secteur inhabituel mais historiquement prisées par des profils d'ingénieurs statisticiens.

En l'absence de données historiques suffisantes (cold start problem, particulièrement aigu en contexte camerounais), S_collab est initialisé à la moyenne des similarités MEPC entre le métier visé par le candidat et les métiers correspondant à l'offre, ce qui constitue un prior structuré préférable à une valeur nulle.

3.8 PIPELINE DE TRAITEMENT ET D'INTEGRATION DES DONNEES

3.8.1 Vue d'ensemble du pipeline ETL

Le pipeline complet d'intégration des données est structuré en six phases séquentielles, chacune alimentant la suivante. Les phases 1 et 2 sont exécutées une seule fois (ingestion des référentiels statiques) ; les phases 3 à 6 sont exécutées de manière incrémentale lors de chaque mise à jour du corpus d'offres ou de candidats.

Phase	Nom	Opérations principales	Outils	Output
P1	Ingestion ESCO	Chargement des 18 CSV ; contraintes unicité ; index fulltext ; marquage collections (digital, green, transversal)	Python + LOAD CSV Neo4j	16 types nœuds Neo4j
P2	Ingestion MEPC/NCF	Chargement des Excel ; mapping MEPC↔ISCO via codes_citp ; relations CONTIENT hiérarchiques	Python openpyxl + Cypher	Hiérarchies MEPC/NCF dans Neo4j
P3	Normalisation offres	Nettoyage textes ; extraction compétences (LLM) ; mapping vers ESCO URIs (pgvector ANN)	Python Pandas + LLM API + Sentence-Transformers	Offres normalisées + URIs ESCO
P4	Normalisation candidats	Mapping qualification→MEPC ; mapping filière→NCF (LLM) ; extraction compétences déclaratives	Python + LLM few-shot	Candidats avec codes MEPC/NCF
P5	Génération embeddings	Embedding de toutes les entités (skills, métiers, offres, candidats) ; insertion pgvector HNSW	Sentence-Transformers + psycopg2	Table embeddings + index HNSW
P6	Calcul score hybride	Requêtes ANN pgvector (top-50) → filtrage Cypher (skill gap) → scoring hybride → roadmap LLM	Python + Neo4j driver + pgvector + LLM	Table recommandations + roadmap JSONB

Tableau 3.5 — Pipeline ETL en six phases du système de recommandation

3.8.2 Gestion de la qualité des données

La qualité des données est un enjeu critique dans notre contexte. Plusieurs stratégies de contrôle sont déployées :

- **Détection des doublons** : les contraintes UNIQUE sur les conceptUri ESCO et les codes MEPC/NCF garantissent l'idempotence de l'ingestion ; toute tentative de duplication est silencieusement ignorée (MERGE).
- **Validation du mapping LLM** : les URIs ESCO proposées par le LLM sont validées par une requête de confirmation Neo4j (MATCH (s:Compétence {conceptUri: \$uri}) RETURN

count(s) > 0 AS valid). Toute URI invalide déclenche une recherche de fallback par pgvector ANN.

- **Seuil de confiance sur les embeddings** : seules les relations SIMILAIRE_A avec un score cosinus ≥ 0.82 sont persistées dans Neo4j, évitant de créer des faux positifs sémantiques.
- **Champ text_to_embed non vide** : une contrainte de non-nullité sur text_to_embed garantit qu'aucun vecteur nul ou aléatoire n'est inséré dans la table embeddings.

3.9 STACK TECHNIQUE COMPLETE ET JUSTIFICATION DES CHOIX

3.9.1 Vue d'ensemble de la stack

Composante	Technologie retenue	Version	Justification synthétique	Alternatives écartées
Base de données graphe	Neo4j Community	5.x	Leader marché, Cypher déclaratif, APOC gratuit, déploiement local	Neptune (cloud payant), ArangoDB
Base vectorielle	pgvector + PostgreSQL	16.x + 0.7.0	Co-localisation métadonnées+vecteurs, SQL standard, HNSW intégré	Pinecone, Weaviate, Milvus
Modèle d'embedding	paraphrase-multilingual-mpnet-base-v2	2023	Multilingue FR/EN, 768 dims, gratuit, local, optimisé similarité	CamemBERT (monolingue), text-embedding-3 (payant)
LLM extraction/génération	GPT-4o / Mistral Large	2024	RAG structuré, JSON mode, multilingue, qualité extraction NER	Llama3 local (qualité moindre en FR)
Orchestration pipeline	Python 3.11 + Apache Airflow	3.11 / 2.9	Écosystème ML natif, DAGs Airflow pour jobs planifiés	Luigi, Prefect
Traitement données	Pandas + Polars	2.x / 0.x	Pandas pour prototypage, Polars pour les grandes tables (>1M lignes)	Spark (surdimensionné)
Framework NLP	Sentence-Transformers + spaCy-fr	3.x	SentenceTransformers pour embeddings, spaCy pour pré-traitement FR	HuggingFace Transformers seul
API de recommandation	FastAPI + Uvicorn	0.115	Asynchrone, typage Pydantic, documentation OpenAPI auto-générée	Flask (synchrone), Django (lourd)
Drivers base de données	neo4j-driver + psycopg2	5.x / 2.9	Drivers officiels, support async, transactions bolt protocol	—
Tests	pytest + pytest-asyncio	8.x	Fixtures, paramétrisation, couverture ; asyncio natif pour FastAPI	unittest (verbeux)

Tableau 3.6 — Stack technique complète du système et justification des choix

3.9.2 Architecture de déploiement et contraintes de ressources

Le système est conçu pour fonctionner sur une infrastructure modeste, alignée sur les contraintes matérielles réalistes du contexte camerounais. La configuration minimale validée est :

-
-
-
-

Cette configuration est intentionnellement sobre. Un système de recommandation déployable uniquement sur infrastructure cloud coûteuse n'aurait aucune utilité pratique pour le marché camerounais. La décision d'utiliser pgvector (dans PostgreSQL existant) plutôt qu'une base vectorielle dédiée, et Neo4j Community (gratuit) plutôt qu'Enterprise, découle directement de cette contrainte de déployabilité.

3.10 PROTOCOLE D'ÉVALUATION DU SYSTÈME

3.10.1 Métriques de performance

L'évaluation d'un système de recommandation est multi-dimensionnelle. Quatre familles de métriques sont retenues, correspondant aux quatre dimensions pertinentes pour notre jury de mémoire :

Dimension	Métrique	Définition	Valeur cible
Précision	Precision@k	Proportion d'offres pertinentes parmi les k premières recommandations	$P@5 \geq 0.70$
Rappel	Recall@k	Proportion d'offres pertinentes retrouvées parmi toutes les offres pertinentes	$R@10 \geq 0.60$
Classement	NDCG@k	Normalized Discounted Cumulative Gain — pénalise les bonnes recommandations mal classées	$NDCG@5 \geq 0.75$
Skill gap	Gap Coverage Rate	Proportion de compétences manquantes correctement identifiées	$GCR \geq 0.65$
Explicabilité	Faithfulness (RAGAs)	Score de fidélité de la roadmap aux données du graphe (0-1)	≥ 0.80
Diversité	Intra-list Diversity	Diversité sémantique des k recommandations (éviter les répétitions)	$ILD \geq 0.30$
Latence	Temps réponse P95	95e percentile du temps de recommandation complet	< 2 secondes

Tableau 3.7 — Métriques d'évaluation du système de recommandation

3.10.2 Protocole d'évaluation et constitution du jeu de test

L'évaluation suit un protocole d'expert judgment pooling, standard dans l'évaluation des systèmes de recherche d'information quand aucune vérité terrain (ground truth) n'existe a priori. Trois experts du marché du travail camerounais (un conseiller MINEFOP, un DRH d'entreprise, un consultant emploi) évaluent la pertinence de 200 paires (candidat, offre) en utilisant une échelle de Likert à 4 niveaux (non pertinent, peu pertinent, pertinent, très pertinent). Ces jugements constituent la vérité terrain pour le calcul de Precision@k et NDCG@k.

La comparaison est menée entre trois conditions expérimentales : (1) le système hybride complet (graphe + vecteur + collaboratif), (2) un système ablation sans composante graphe (vecteur + collaboratif uniquement), et (3) un baseline de matching par mots-clés TF-IDF représentant l'état de l'art implicite des plateformes d'emploi locales actuelles. L'analyse des résultats permettra de

quantifier l'apport marginal de chaque composante, ce qui constitue la contribution scientifique principale de ce mémoire.

3.10.3 Reproductibilité et mise à disposition

L'ensemble du code source (pipeline d'ingestion, scripts Cypher, schéma pgvector, module de scoring hybride, prompts LLM) sera mis à disposition dans un dépôt GitHub public sous licence MIT à la date de soutenance. Les fichiers ESCO utilisés sont disponibles sous licence CC-BY 4.0 sur le portail ESCO officiel. Les données MEPC et NCF sont disponibles sur le site de l'INS Cameroun (statistics-cameroon.org). Les données empiriques d'offres et de candidats sont anonymisées avant publication conformément aux exigences de protection des données personnelles.

3.11 SYNTHÈSE ET POSITIONNEMENT DE LA CONTRIBUTION

3.11.1 Récapitulatif des choix méthodologiques structurants

Ce chapitre a exposé une méthodologie construite autour d'une conviction centrale : les problèmes de matching emploi-compétences sont fondamentalement des problèmes d'hétérogénéité sémantique, et aucune technique unique — ni la correspondance de mots-clés, ni les embeddings seuls, ni le graphe seul — ne peut les résoudre de manière satisfaisante. La contribution proposée est précisément cette hybridation raisonnée et formellement définie.

Synthèse des apports originaux

1. **INTÉGRATION TRIPARTITE** : Première formalisation d'un graphe de connaissances croisant ESCO, MEPC (camerounaise) et NCF (camerounaise) dans une architecture Neo4j unifiée. 2. **SCORE HYBRIDE CONTEXTUEL** : Formulation d'un score $S(c,o) = \alpha \cdot S_{\text{sém}} + \beta \cdot S_{\text{graph}} + \gamma \cdot S_{\text{collab}}$ calibrable, avec une composante graphe fondée sur la traversée de la hiérarchie ESCO. 3. **PIPELINE LLM-ANCRÉ** : Utilisation du LLM exclusivement en mode RAG (sur les référentiels chargés en contexte) pour éliminer les hallucinations sur les nomenclatures. 4. **DÉPLOYABILITÉ LOCALE** : Architecture fonctionnelle sur infrastructure modeste (16 Go RAM, CPU uniquement), sans dépendance à des services cloud payants.

3.11.2 Limites identifiées et perspectives

Cette approche présente des limites qui méritent d'être énoncées avec rigueur :

4. **Cold start problem** : la composante collaborative est non-fonctionnelle à froid, c'est-à-dire pour les nouveaux candidats ou les nouvelles offres sans historique d'interaction. Le prior MEPC proposé atténue mais ne résout pas totalement ce problème.
5. **Couverture du corpus d'offres** : le corpus d'offres est biaisé vers les plateformes numériques, sous-représentant le marché informel et les offres transmises par réseaux relationnels, qui constituent une part significative du marché de l'emploi camerounais.
6. **Qualité du mapping NCF/MEPC automatique** : le mapping LLM entre les textes libres des candidats et les codes officiels NCF/MEPC introduit une erreur mesurable (estimée à ~15% d'attributions incorrectes sans validation humaine). Une interface de correction humaine dans la boucle (human-in-the-loop) est recommandée en production.
7. **Évolution des référentiels** : ESCO est mis à jour annuellement ; MEPC (2013) et NCF (2017) n'ont pas encore connu de révision. Un plan de maintenance du graphe intégrant ces mises à jour est nécessaire pour la pérennité du système.

Ces limites constituent autant de perspectives de recherche futures. En particulier, l'extension du système à la couverture du marché informel via des techniques de web scraping des réseaux sociaux professionnels africains (LinkedIn Afrique, Jobberman, Africawork) et l'intégration d'un module d'apprentissage en ligne (online learning) pour adapter les pondérations α , β , γ en temps

réel selon les retours utilisateurs constituent les prochaines étapes naturelles de ce programme de recherche.

En conclusion, la méthodologie présentée dans ce chapitre répond à l'ambition formulée dans l'objectif général : concevoir un système de recommandation hybride emploi-compétences capable d'identifier les écarts de compétences, de classer les correspondances avec les offres d'emploi et de proposer une roadmap personnalisée d'amélioration du profil. Chaque décision technologique — Neo4j pour le raisonnement structurel, pgvector pour la sémantique dense, le LLM pour l'enrichissement et l'explication — est justifiée par des arguments à la fois théoriques et pragmatiques, ancrés dans les contraintes réelles du marché du travail camerounais.